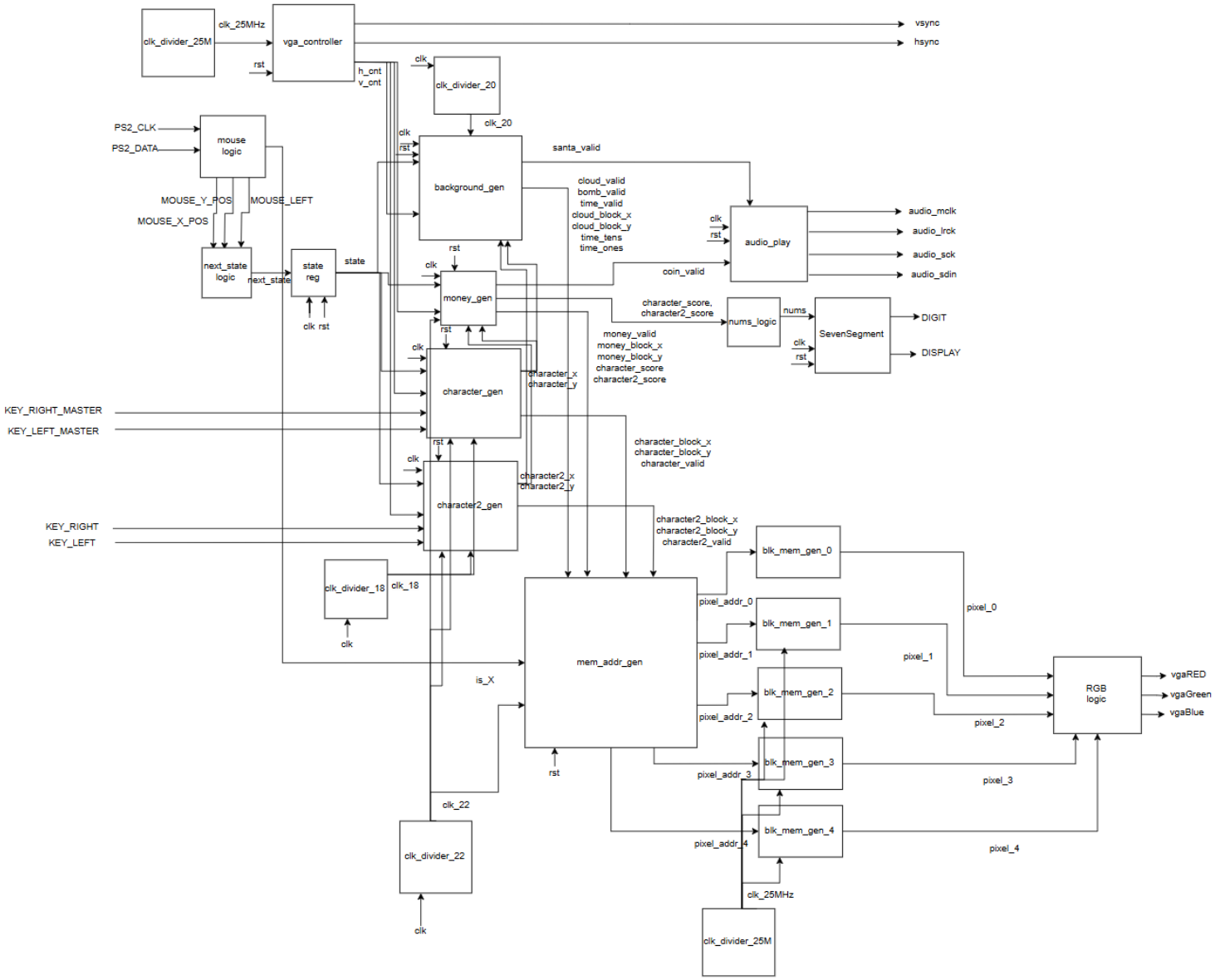
Block diagram

1. 主要 master 板



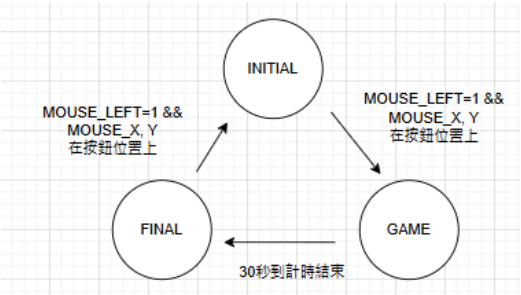
[畫面顯示]

- 遊戲中所有移動中的元素都有各自的 clk，以達到最自然的動畫效果。money, character(2)使用 clk_22，background（掉落物）使用 clk_20，character 在被掉落物壓住時使用 clk_18。
- H_cnt, v_cnt 會輸入 Background_gen, Money_gen, character_gen，這幾個 module 用來產生會擺放各遊戲元素的 x, y 座標位置，並在要擺放的(h, v_)cnt 處 valid=1。
- Background 中還會進行遊戲倒數計時，並將目前倒數的數字傳到 mem_addr_gen 擷取需要顯示的數字圖片。

- KEY_LEFT, KEY_RIGHT 是鍵盤按下左右（詳見 slave 板說明），會傳至 character_gen 去移動角色。
- Character_X, character_Y 也會傳給 money_gen，在 money_gen 中判斷角色位置與金幣位置是否重疊。Money_gen 中會計算兩個角色的金幣分數，傳到 sevenSegment 顯示。
- Character_X, character_Y 也會傳給 background_gen，在 background_gen 中會判斷是否掉落物位置有壓到角色位置，若有則傳 character_dead 給 character，改變角色 y 方向位置讓角色往下被壓。
- 所有元素的 x, y 位置、valid 信號會傳到 mem_addr_gen，mem_addr_gen 會傳給 mem_gen 當下是 valid 的元素所對應的 x, y 位置，其在圖片中的 address。
- Mem_gen 是額外開的記憶體，會由 address 生成正確 pixel 的顏色，傳到 rgb_logic。
- Rgb_logic 中會判斷這五塊 mem 的 pixel 誰要優先顯示，也就是疊圖時誰會顯示在誰前面，最終只挑最前面的物件顯示至{rgb_red, rgb_green, rgb_blue}。

State 轉換和滑鼠

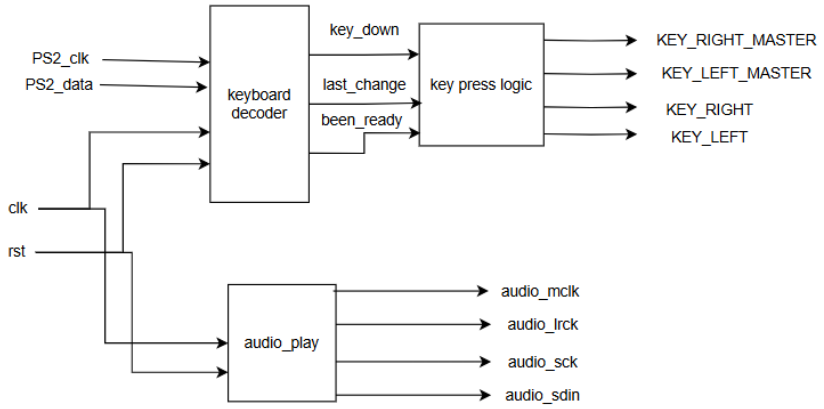
- 滑鼠使用助教提供的模板，不多做內部解釋。MOUSE_LEFT=1(按下滑鼠時)，判斷若 MOUSE_X, MOUSE_Y 在按鈕範圍，則按紐按下，轉 state。
- 以下是 state 圖：



配樂

- 主板子只負責 GAME 的音效，從 money_gen 傳 coin_valid、background_gen 傳 santa_valid 到 audio_play，代表吃到金幣、被壓到。
- 在音效 valid 時，output 音效信號。

2. Slave 板



- 處理兩玩家在鍵盤按下的左右移動，並以 KEY_RIGHT(_MASTER), KEY_LEFT(_MASTER)輸出給主要板子。有加 MASTER 為玩家一。
- Audio_play 會讀遊戲配樂的.mem 檔，並以 audio 信號輸出。

Partial code screenshot with the explanation:

1.

```
if(key_down[last_change] && !key_pressed2)begin
    now_last_change2<=last_change;
    case(last_change)
        HOME_CODES:begin KEY_LEFT_MASTER<=1; pass<=1;key_pressed2<=1;end
        END_CODES:begin KEY_RIGHT_MASTER<=1;pass2<=1; key_pressed2<=1;end
    endcase
end else if(key_down[now_last_change2]==1'b0 && key_pressed2)begin

    case(now_last_change2)
        HOME_CODES:begin KEY_LEFT_MASTER<=0;pass<=0;key_pressed2<=0;end
        END_CODES:begin  KEY_RIGHT_MASTER<=0; pass2<=0;key_pressed2<=0;end
    endcase
end
```

以上是寫在 slave 板子上的 code，最後會將 KEY_LEFT_MASTER 和 KEY_RIGHT_MASTER 傳回 master 板子。

以上是其中一個角色的鍵盤判斷，為了讓兩個角色可以同時使用鍵盤，而不會因為其中一人按下鍵盤導致另一人無法按，所以要分兩個 now_last_change，now_last_change2 跟 key_pressed、key_pressed2。因此會有兩個 always，但以上指展是一個。

在同一個鍵盤上操作兩個角色，一個人是 A(A_CODES)、D(D_CODES)，另一個人是數字鍵 4(HOME_CODES)、6(END_CODES)。

```
if(KEY_RIGHT   && character2_x<=608)begin
  pass<=1;pass2<=0;
    character2_x<=character2_x+8;
end else if(KEY_LEFT  && character2_x!=0)begin
  pass<=0;pass2<=1;
    character2_x<=character2_x-8;
end
else begin pass<=0;pass2<=1;end
```

以上是 master FPGA 接收 slave FPGA 板子傳來的 KEY_RIGHT 和 KEY_LEFT 訊號，這兩個 always 加起來就會讓玩家按住鍵盤時，角色會一直前進。除此之外，我們的角色也會有連貫的動作，在第 6 點會詳細說明。

2.

```
initial begin
    $readmemh("audio_data.mem", rom); // Load audio data from memory file
     $readmemh("audio_santa3.mem", rom_santa);
end
```

以上將.wav 檔轉為.MEM 檔，再將音樂的檔案讀入，這樣可以讓音樂聽起來比起一般使用 note_gen module 聽起來還要更順暢。

```
audio_out <= rom[rom_addr];
rom_addr <= (rom_addr == 59999-1) ? 0 : rom_addr + 1; // Loop playback
```

以上是無限重播音樂，當 rom_addr 到達音檔的最後一個資料時，將 rom_addr 重設為 0 繼續播放音樂。

```
always@(*)begin
 if(coin_valid && !hh)begin
     coin_valid_cur=1;
 end else if(hh) coin_valid_cur=0;
end
```

以上是因為判斷 coin_valid 的週期比決定播放音符的周期短，所以為了解決沒有播放到音樂的問題，多增加了以上這個判斷，確保每一個 coin_valid==1 時都會被播放到。

```
if(now_coin)begin
    hh<=0;
    audio_out <= rom[rom_addr]*volume;
    rom_addr <= rom_addr + 1; // 更新地址
    if (rom_addr == 25001 - 1) begin
        // 播放結束
        playing <= 0;
    end
end
```

以上是當 coin_valid==1 時，只播放一次音樂。

3.

```
coin_valid<=0;
        for (i = 0; i < money_COUNT; i = i + 1) begin
          if (money_y[i] == -32) begin
              money_eat[i]<=0;
          end

            if ((character_x>money_x[i]-20 && character_x<money_x[i]+20 && character_y>money_y[i]-20 && character_y<money_y[i]+20)||(character2_x>money_x[i]-20 &&
character2_x<money_x[i]+20 && character2_y>money_y[i]-20 && character2_y<money_y[i]+20))begin
              if(money_eat[i] == 0)begin
                coin_valid<=1;
                money_eat[i] <= 1;
                if(character2_x>money_x[i]-20 && character2_x<money_x[i]+20 && character2_y>money_y[i]-20 && character2_y<money_y[i]+20)begin
                    character2_score<=character2_score+1;
                end else begin
                    character_score<=character_score+1;
                end
              end else begin
                money_eat[i] <= 1;
              end
            end
        end
```

以上是判斷角色是否有吃到金幣，如果有就將 money_eat 設為 1;以上也是判斷 character_score 的部分，如果角色在金幣的範圍內且金幣還沒有被吃掉，則將 character_score 的分數加 1。

```
if (h_cnt >= money_x[0] && h_cnt < money_x[0]+32 && v_cnt >= money_y[0]  && v_cnt < money_y[0]+32 && money_eat[0]!=1) begin
      money_valid=1; money_block_x=money_x[0];  money_block_y=money_y[0]; end
```

以上是判斷 h_cnt 和 v_cnt 等於多少時 money_valid==1，當 money_valid==1 時，就會讓 pixel_addr 顯示 money 的圖案。

4.

```
character_dead<=0;
for(j=0;j<CLOUD_COUNT;j=j+1)begin
  if(character_x>cloud_x[j]-30 && character_x<cloud_x[j]+30 && character_y>cloud_y[j]-30 && character_y<cloud_y[j]+30)begin
    pass3<=1; character_dead<=1;
end
end
```

以上是判斷角色是否在掉落物的範圍內，如果在就將 character_dead 設為 1。

```
if(character2_dead)begin
    character2_y<=character2_y+1;
end
```

以上是當 character_dead 為 1 時，角色以 clk_18 的速度往下移動。

5.

```
for (i = 0; i < CLOUD_COUNT; i = i + 1) begin
          if (cloud_y[i]-480 >= 0) begin
            // 如果雲完全移出屏幕，將其重置到屏幕頂部
            cloud_y[i] <= -32;  // 假設屏幕高度為 480
          end else begin
            // 雲向上移動
            cloud_y[i] <= cloud_y[i] + 1;
          end
        end
```

以上是為了讓背景的圖案不斷在螢幕上捲動，如果大於 480 就將它重設為-32，而非 0 這樣他才不會像雲朵憑空出現在螢幕最上排，而是像捲回來一樣重新出現。但是因為這邊用到負號，因此要將變數設為 signed。

6.

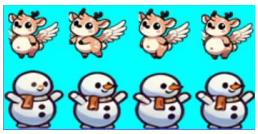
```
if(pixel_4 == 12'b0)begin
            if(time_valid && pixel_2!=12'b0) {vgaRed, vgaGreen, vgaBlue} =pixel_2;
            else if((character_valid||character2_valid) && pixel_0!=12'b0000_1111_1111) {vgaRed, vgaGreen, vgaBlue} =  pixel_0;
            else if((bomb_valid||cloud_valid) && pixel_1!=12'b0000_1111_1111) {vgaRed, vgaGreen, vgaBlue} =  pixel_1;
            else if(money_valid && pixel_3!=12'b0000_1111_1111) {vgaRed, vgaGreen, vgaBlue} =  pixel_3;
            else {vgaRed, vgaGreen, vgaBlue} = 12'b0000_0010_0110;
          end
          else {vgaRed, vgaGreen, vgaBlue} = 12'b1110_1110_1110;
```

以上是 GAME state 處理要顯示哪一個 pixel，其中 pixel_4 是一張下雪的圖，所以如果 pixel_4 是黑色(12’b0)，則顯示其他應該顯示的圖，否則顯示灰色(12’b1110_1110_1110)。透過 time_valid、character_valid、bomb_valid、cloud_valid、money_valid 是否等於 1，來判斷現在應該顯示哪一個 pixel，除此之外，如果現在是該顯示的圖的顏色是被景色(12'b0000_1111_1111)就顯示深藍色(12’b0000_0010_0110)。疊圖的順序會依照 if-else 的優先順序來疊圖，舉例來說，GAME state 的

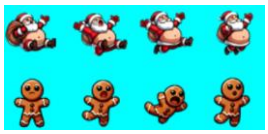
疊圖順序由最上層到最下層，分別是 pixel_4 的雪、pixel_2 的 time、pixel_0 的 character、pixel_1 的 bomb 和 cloud、pixel_3 的 money，最後是背景的深藍色(12’b1110_1110_1110)。

(pixel_4)

(pixel_2)



(pixel_0)



(pixel_1)

(pixel_3)

```
if(enable_mouse_display) begin
          {vgaRed, vgaGreen, vgaBlue} = mouse_pixel;
        end
        else if((h_cnt>=293 && h_cnt<352 && v_cnt>=208 && v_cnt<272) || (h_cnt>=293 && h_cnt<352 && v_cnt>=320 && v_cnt<352)) begin
          if(pixel_2==12'b0000_1111_1111) {vgaRed, vgaGreen, vgaBlue} = 12'b0;
          else {vgaRed, vgaGreen, vgaBlue} = pixel_2;
        end
        else {vgaRed, vgaGreen, vgaBlue} = 12'b0;
```

以上是 INIT state 處理要顯示哪一個 pixel，其中 enable_mouse_display=1 時要顯示滑鼠游標，否則當範圍在特定範圍時就要顯示 pixel_2，否則顯示黑色(12’b0)。

7.

```
always @(posedge clk_25MHz) begin
  move<=(move+32)%160;
end
```

```
pixel_addr_ =(h_cnt-cloud_block_ +move)+(v_cnt-cloud_block_ +32)*128;
```

我們角色(pixel_0)和金幣(pixel_3)還有掉落物(pixel_1)，都會也不同的動態動作，以 pixel_3 來舉例，pixel_addr 根據 clk_25MHz 來加上 move，達到金幣翻轉的狀態。

8.

```
if(h_cnt>=288 && h_cnt<352 && v_cnt>=208 && v_cnt<272)begin
  pixel_addr_2 = (h_cnt-288) + (v_cnt-48) *128;
end
```

如果要顯示聖誕老人，因為聖誕老人在 pixel_addr_2 的 h_cnt=0~64、v_cnt=32*5=160~224，所

以如果要讓聖誕老人顯示在螢幕的 h_cnt=288~352、v_cnt=208~272，就要 pixel_addr_2=(h_cnt-288)+(v_cnt-208_160)*128。

(pixel_2)

9.

```
assign isX = (MOUSE_X_POS>296 && MOUSE_X_POS<351 && MOUSE_Y_POS>325 && MOUSE_Y_POS<342)? 1:0;
```

```
else if(h_cnt>=288 && h_cnt<352 && v_cnt>=320 && v_cnt<352)begin
  if(isX) pixel_addr_2 = (h_cnt-224) + (v_cnt-256) *128;
  else pixel_addr_2 = (h_cnt-288) + (v_cnt-256) *128;
end
```

如果 isX 為 1，則顯示圖片中較深較小的 Start 或 Next，反之顯示邊較大的 Start 或 Next，達到圖片有被按下去的感覺。

```
FINAL:begin
  if(MOUSE_LEFT && isX) begin
    next_state = INITIAL;

  end
end
```

```
INITIAL:begin
    if(MOUSE_LEFT && isX && cooldown_done) begin
      next_state=GAME;
      start_game=1;
    end
end
```

```
always @(posedge clk or posedge rst) begin
      if (rst) begin
          cooldown_counter <= 0;
      end else if (state == INITIAL && cooldown_counter > 0) begin
          cooldown_counter <= cooldown_counter - 1; // 冷卻計數
      end else if (state == FINAL && next_state == INITIAL) begin
          cooldown_counter <= 27'd100_000_000; // 設定冷卻時間
      end
  end
```

以上三張是分別是 FINAL 和 INIT 轉 state 的條件，因為 FINAL 轉成 INIT 的條件跟 INIT 轉成 GAME 的條件一樣，所以會從 FINAL 直接轉到 GAME，因此要在 state==FINAL && next_state==INITIAL 時將 cool_down 設為 0，然後在 state==INITIAL 時開始倒數計時，然後確保 state 為 INITIAL 且倒數計時計時倒 0 時才可以將 state 從 INITIAL 轉為 GAME，以防 state 直接從 FINAL 轉乘 GAME，

達到讓 state 停留在正確 state。

10.

```
always @(posedge clk_ori or posedge rst) begin
    if (rst) begin
      counter <= 30;  // 初始化為 30
      cnt<=0;         秒
    end else begin
      if(start_game) counter <= 30;
      else begin
        if(cnt<28'd100_000_00 ) cnt<=cnt+1;
        else begin
          if (counter>=1) begin
            counter <= counter - 1;
            cn <=0;
t         end
          else counter <= counter;
        end
      end
    end
  end
```

```
if(time_ten ==0 && time_ones==0) next_stat =FINAL;
```

以上是在每次 start_game==1(state 轉為 GAME)時，將倒數計時器初始為 30 秒，然後計算開始倒數計時，然後在 state 為 GAME 時，如果時間的時位數(time_ten)和時間的個位數(time_ones)皆為 0 時，也就是倒數計時為 0 時，就將 next_state 設為 FINAL。

Problem Encountered :

1. 問題:因為使用鍵盤讓腳色上下左右移動會耗費過多的 LUT

   解決方法: 猜測是因為二維運動會耗費 LUT，因此從二維運動改為左右運動(一維)，若掉落物碰到角色，角色會被掉落物往下壓。

2. 問題:若完整讀一整首歌，也會因會歌曲的檔案過大而耗費過多 LUT

   解決方法:使用短短音效重複回播

3. 問題:原本預計要差一個滑鼠和兩個鍵盤，但 USB 插孔不足

   解決方法:所以取消兩個鍵盤改用一個鍵盤和一個滑鼠

4. 問題:本來希望兩個 sevenSegment 可以分別顯示兩個腳色的分數，但是發現如果要傳分數會需要很多排線，所以改成在同一個 sevenSegment 的左右

   解決方法: 改成在同一個 sevenSegment 的左邊顯示 character2(雪人)的分數，右邊顯示 character(麋鹿)的分數。

5. 問題:疊圖的時候看起來像方塊的疊圖，而非角色跟角色疊圖

   解決方法:將會有疊圖問題的圖放在不同的 memory_gen 裡

6. 問題:隔空吃金幣的問題

   解決方法:發現是時間差的問題，因此將 clk 改為較短周期的 clk

7. 問題:背景捲圖時，要在對的位子重新出現，一開始設-32 會捲完一次之後，隔一陣子再捲第二次

   解決方法:後來發現是因為將變數設為-32，但是卻沒有將變數設為 signed，所以才會是錯的捲圖，後來加上 signed 就解決了。

8. 問題:因為從 FINAL state 轉為 INIT state 的判斷條件一樣，所以會直接從 FINAL(經過 INIT)直接轉成 GAME state

   解決方法:設一個冷卻時鐘，讓在 INIT 的判斷需要先等待一定時間，才能轉到下一個 state。